

In the example above, FormBuilder creates a Control Group (which represents a form and input controls), using specifications supplied as arguments. You can add one or more Validators to this specification.

Minimum length validation

```
this.form = fb.group({ 'name': ['', Validators.required,
Validators.minLength(4)] });
```

Maximum length validation

```
• constructor(private fb: FormBuilder){
• this.form = fb.group({ 'name': ['', Validators.required,
Validators.maxLength(4)]}); }
```

Combining Multiple Validators

The Validators class provides the 'compose' method to allow the user to specify multiple validators to a control.

```
• constructor(private fb: FormBuilder){
• this.form = fb.group({ 'name': ['', Validators.compose(
[Validators.required, Validators.maxLength(6)]),}); }
```

Custom validations

The Angular Forms module allows you to create a custom class to validate your input. Note that the validation method is static and that it returns a Validation Result only when there is an error. If everything is ok, this method returns a null. This custom class can be used when specifying the field in the FormBuilder and it can also be used in the Component Template to provide a visual cue.

Example - app.ts

```
• export interface ValidationResult {[key:string]:boolean;}
•
• export class MakeValidator {
•   static isNotMercedes(control: Control): ValidationRe-
sult {
•     console.log(control.value);//Invalid
•     If (control.value == 'mercedes'){
•       Return {'isMercedes' : true;};//Valid
•       return null;}}
• constructor(private fb: FormBuilder){
```